

Sampling — From Zero to Problem-Solving

A plain-language guide to Lecture 5 (Sampling), built to get you directly to solving aliasing / Nyquist / reconstruction problems — including the kind in your assignment.

Who this is for: you don't need to know sampling coming in. Part 1 builds the intuition from scratch using plain language (skip the heavy derivations — you just need to trust the results). Part 2 gives you a formula sheet. Part 3 gives you step-by-step **templates** for the exact problem types you'll be asked to solve, each with a worked example. Part 4 has practice problems (no answers) so you can test yourself.

PART 1 — Building the Intuition (in plain English)

1. What problem are we even solving?

You have a real, continuous signal $x(t)$ — like a sound wave. A computer can't store an infinite, continuous curve, so it takes snapshots ("samples") of the value at evenly spaced instants, every T seconds. The big question this whole lecture answers: **if all I keep is those snapshots, have I lost information, or can I perfectly rebuild the original signal from just the samples?**

Answer: it depends on how *fast* the signal wiggles compared to how *often* you sample it. That's the whole lecture, dressed up in math.

2. The key vocabulary (know these cold)

Term	Plain-English meaning
$x(t)$	The original continuous-time signal.
$X(j\omega)$	Its spectrum (Fourier Transform) — which frequencies it's made of, and how much of each.
Band-limited	The signal contains NO frequency content above some maximum ω_M (its spectrum is exactly zero outside $[-\omega_M, \omega_M]$).
Sampling period T	The time gap between samples (seconds).
Sampling frequency $f_s = 1/T$ ($\omega_s = 2\pi/T$)	How many samples per second you take (or in rad/s).
$x[n] = x(nT)$	The n -th sample — just the value of $x(t)$ at time nT .
Nyquist rate	The MINIMUM sampling frequency that still lets you perfectly recover $x(t)$: $\omega_s = 2\omega_M$ (i.e. $f_s = 2f_M$).
Aliasing	What happens when you sample TOO SLOWLY (below the Nyquist rate) — a high frequency "disguises itself" as a lower frequency.

3. Why sampling = "multiply by a comb of spikes"

Picture $x(t)$ as a smooth wiggly curve. To "sample" it, imagine a row of infinitely thin, infinitely tall spikes (impulses) spaced T seconds apart — called an **impulse train** $p(t)$. If you multiply $x(t)$ by $p(t)$, every spike "picks up" the value of $x(t)$ exactly where it stands and zeroes out everything in between. The result, $xp(t)$, is a row of spikes whose heights are exactly your samples $x(nT)$. That's literally what sampling is, mathematically.

4. The one picture that explains everything

Here's the punchline the whole lecture builds toward, and it's the only picture you actually need to solve problems:

Multiplying in TIME domain corresponds to convolving in FREQUENCY domain. Convolving a spectrum with a train of impulses just **stamps down a copy of that spectrum at every impulse location** — no distortion, just copy-paste.

The impulse train $p(t)$, spaced T apart in time, turns out (via Fourier series) to ALSO be an impulse train in frequency — spaced $\omega_s = 2\pi/T$ apart. So:

Sampling in time $\rightarrow$ copies of $X(j\omega)$, repeated every ω_s , scaled by $1/T$

That's it. That's the entire theorem, visually: take your signal's spectrum (some blob between $-\omega_M$ and $+\omega_M$), and photocopy it every ω_s rad/s, forever, in both directions. **Everything else in the lecture is either proving this rigorously, or dealing with the consequences of it.**

5. The consequence: do the copies overlap?

Each copy of $X(j\omega)$ is $2\omega_M$ wide (from $-\omega_M$ to $+\omega_M$ around its center). The copies are spaced ω_s apart. Obviously:

- If ω_s is BIG ENOUGH that the copies don't touch each other $\rightarrow$ you can put a low-pass filter around just the middle copy and get back an exact, undistorted $X(j\omega)$. **Signal is perfectly recoverable.**
- If ω_s is TOO SMALL, the copies overlap and blend into each other at the edges. Once they've blended you can never again tell where one copy ends and the next begins — the original shape is gone forever. **This is aliasing — information is permanently destroyed.**

The exact boundary between these two cases is the **Nyquist-Shannon Sampling Theorem**: you need $\omega_s > 2\omega_M$ (equivalently $f_s > 2f_M$). The magic number $2\omega_M$ is called the Nyquist rate.

6. What aliasing actually looks like for a single tone

This is the part your assignment is directly testing, so read carefully. Say you sample a pure cosine of frequency f at rate f_s . Because the spectrum repeats every f_s (that's the copies from Section 4), a frequency f produces EXACTLY the same set of samples as $f + f_s$, $f + 2f_s$, $f - f_s$, etc. Also, because cosine is symmetric ($\cos$ is an even function — $\cos(-\theta) = \cos(\theta)$), sampling can't tell $+f$ apart from $-f$ either.

So the full family of frequencies that all look identical after sampling at rate f_s , starting from f , is:

$f + k \cdot f_s$ and $-f + k \cdot f_s$, for any integer k

Each member of this family is called an **alias** of the others. If you only keep the samples, there is fundamentally no way to know which member of the family the signal originally was — they all produce bit-for-bit identical

sample values. This is exactly what your alias.py assignment demonstrates numerically.

7. Reconstruction — turning samples back into a continuous signal

Once $\omega_s > 2\omega_M$, recovery is theoretically just "apply an ideal low-pass filter to keep only the middle copy." But an ideal filter is physically unrealizable (its impulse response is an infinite sinc function). In practice:

- **Zero-Order Hold (ZOH):** just hold each sample's value flat until the next sample arrives — a "staircase" signal. Simple, used in real DACs, but imperfect (lets some high-frequency leakage through, attenuates the passband slightly).
- **Linear interpolation (First-Order Hold):** draw a straight line connecting each pair of adjacent samples. Smoother, closer to the ideal filter's behavior, less high-frequency leakage than ZOH.
- **Ideal LPF (sinc interpolation):** the theoretical best — cutoff exactly at $\omega_s/2$ — but requires an infinite, non-causal filter so it can't be built exactly; it's the benchmark the practical methods are compared against.

PART 2 — Formula Sheet (the only page you need in the exam room)

Quantity / Concept	Formula
Sampling frequency $\leftrightarrow$ period	$f_s = 1/T$, $\omega_s = 2\pi/T = 2\pi f_s$
Sample values	$x[n] = x(nT) = x(n/f_s)$
Nyquist rate (minimum OK rate)	$\omega_s > 2\omega_M$ i.e. $f_s > 2f_M$
Spectrum after sampling	$X_p(j\omega) = (1/T) \sum_k X(j(\omega - k\omega_s))$ — copies of $X(j\omega)$ every ω_s , scaled by $1/T$
Alias family of a tone f (sampled at f_s)	All frequencies of the form $f + k \cdot f_s$ or $-f + k \cdot f_s$, $k \in \mathbb{Z}$, produce identical samples
Lowest positive alias partner of f , given $0 < f < f_s/2$	partner = $f_s - f$ (this is the smallest positive frequency $\neq f$ with identical samples — see Part 3, Template)
Ideal reconstruction filter	$H(j\omega) = T$ for $ \omega < \omega_s/2$, else 0. Impulse response $h(t) = \text{sinc}(t/T)$
Ideal reconstruction formula	$x_r(t) = \sum_n x(nT) \cdot \text{sinc}((t-nT)/T)$
ZOH impulse response / freq.	$h_0(t) = \text{rect over } [0, T]$; $H_0(j\omega) = T \cdot e^{-j\omega T/2} \cdot \text{sinc}(\omega T/2\pi)$
Linear interp. impulse/freq.	$h_1(t) = \text{triangle over } [-T, T]$; $H_1(j\omega) = T \cdot \text{sinc}^2(\omega T/2\pi)$
DTFT of the sampled sequence	$X(e^{j\omega}) = \sum_n x[n] e^{-j\omega n}$ (this is $X_p(j\omega)$ with the substitution $\omega T \rightarrow \omega$)
DFT frequency of bin k (N-point DFT, sample rate f_s)	physical frequency of bin $k = k \cdot f_s / N$ (for $k = 0 \dots N/2$, positive frequencies; $k = N/2+1 \dots N-1$ are wrap)
Frequency resolution of DFT	$\Delta f = f_s / N$

PART 3 — Problem-Solving Templates (with worked examples)

These are step-by-step recipes. When you see a new problem, first figure out which template it matches, then follow the steps mechanically.

Template A — Find the lowest positive alias partner of frequency f , sampled at f_s

Use this when asked: "find another frequency that produces identical samples", "find the alias of f ", or anything matching your alias.py assignment's `lowest_alias_pair(f, fs)`.

Step 1 — Recall the alias family.

Every frequency of the form $f + k \cdot f_s$ or $-f + k \cdot f_s$ (for integer k) produces identical samples to f .

Step 2 — You want the **SMALLEST POSITIVE** member of that family, other than f itself.

Try $k = 1$ on the $-f$ branch first, since that's usually the closest one: $\text{candidate} = -f + f_s = f_s - f$.

Step 3 — Check it's valid: positive, and not equal to f .

Since the problem guarantees $0 < f < f_s/2$, we know $f_s - f > f_s/2 > f$, so $f_s - f$ is positive AND different from f automatically. No other candidate (from either branch) can land strictly between 0 and $f_s - f$, so this is the smallest one.

Step 4 — Answer:

lowest alias partner = $f_s - f$

Worked example

PROBLEM: $f = 300$ Hz, $f_s = 1000$ Hz. Find the lowest positive alias partner.

Solution:

$\text{partner} = f_s - f = 1000 - 300 = 700$ Hz.

Sanity check against the theory: sampling 300 Hz and 700 Hz at 1000 Hz gives identical sample values, because $700 = -300 + 1000$ is in 300's alias family ($k=1$ on the $-f$ branch).

Template B — Is a given sampling rate sufficient (Nyquist check)?

Use this when asked: "is f_s enough to sample $x(t)$ without aliasing?", "what's the minimum sampling rate?", "can this signal be perfectly recovered?"

Step 1 — Find ω_M (or f_M), the highest frequency present in $x(t)$.

If $x(t)$ is a sum of cosines/sines, ω_M is the largest one. If $X(j\omega)$ is given as a plot, read off where it becomes zero.

Step 2 — Compute the Nyquist rate: $2\omega_M$ (or $2f_M$).

Step 3 — Compare to the given ω_s (or f_s).

$\omega_s > 2\omega_M \rightarrow$ safe, perfectly recoverable, no aliasing. $\omega_s \leq 2\omega_M \rightarrow$ aliasing occurs, NOT recoverable.

Worked example

PROBLEM: $x(t) = 3\cos(2\pi \cdot 400t) + 5\cos(2\pi \cdot 900t)$ is sampled at $f_s = 1500$ Hz. Is this enough to avoid aliasing?

Solution:

Highest frequency present: $f_M = 900$ Hz. Nyquist rate = $2 \times 900 = 1800$ Hz.

Given $f_s = 1500$ Hz < 1800 Hz $\rightarrow$ **NOT enough — aliasing will occur.** The 900 Hz component will fold down and appear as a lower alias frequency (use Template A with $f = 900$, $f_s = 1500$ to find exactly where it folds to: $1500 - 900 = 600$ Hz).

Template C — Given a sampled (aliased) frequency, find what it could really be

Use this when asked: "you observe a component at frequency f_{obs} after sampling at f_s — what could the true original frequency have been?" (This is Template A run in reverse / as a set.)

Step 1 — The true frequency f_{true} must alias to f_{obs} , meaning f_{obs} is a member of f_{true} 's family:

$$f_{\text{obs}} = f_{\text{true}} + k \cdot f_s \text{ or } f_{\text{obs}} = -f_{\text{true}} + k \cdot f_s, \text{ for some integer } k.$$

Step 2 — Solve for all plausible f_{true} values (usually the question restricts to a range, e.g. "assume the true frequency was between 0 and f_s ").

Step 3 — If additional info narrows it down (e.g. "the original signal was known to be below $f_s/2$ "), apply that constraint to pick the one true answer.

Worked example

PROBLEM: A signal was sampled at $f_s = 8000$ Hz, and after sampling you observe a tone at 500 Hz. If you're told the true signal frequency was somewhere between 7000 Hz and 9000 Hz, what was it?

Solution:

$f_{\text{obs}} = 500$ must satisfy $500 = f_{\text{true}} + k \cdot 8000$ or $500 = -f_{\text{true}} + k \cdot 8000$ for some integer k .

Try the second form: $f_{\text{true}} = k \cdot 8000 - 500$. For $k = 1$: $f_{\text{true}} = 8000 - 500 = 7500$ Hz — this falls inside [7000, 9000]. **Answer: 7500 Hz.** (Check: 7500's alias family includes $-7500 + 8000 = 500$ Hz. ✓)

Template D — Ideal LPF reconstruction: what filter do you need?

Use this when asked: "design the ideal reconstruction filter for a signal sampled at f_s , band-limited to f_M " or "what cutoff frequency should the LPF have?"

Step 1 — Confirm Nyquist holds: $\omega_s > 2\omega_M$ (if it doesn't, no filter can recover the signal — say so).

Step 2 — Pick ω_c strictly between ω_M and $\omega_s - \omega_M$ (any value works in theory; the conventional/default choice is the midpoint, $\omega_c = \omega_s/2$).

Step 3 — State the filter: $H(j\omega) = T$ for $|\omega| < \omega_c$, and 0 otherwise, where $T = 1/f_s$.

Worked example

PROBLEM: A signal band-limited to $f_M = 3$ kHz is sampled at $f_s = 10$ kHz. Specify the ideal reconstruction filter (using the conventional cutoff choice).

Solution:

Check Nyquist: $\omega_s = 2\pi(10000) > 2\omega_M = 2\pi(6000)$. ✓ Recoverable.

Conventional cutoff: $\omega_c = \omega_s/2 \rightarrow f_c = f_s/2 = 5000$ Hz (any value between 3000 Hz and 7000 Hz would technically also work, since that's the gap between the baseband edge and the next copy's edge).

Filter: $H(j\omega) = T = 1/10000$ s for $|f| < 5000$ Hz, and 0 elsewhere.

Template E — DFT bin $\leftrightarrow$ physical frequency conversion

Use this when asked: "an N-point DFT is taken of a signal sampled at f_s — what physical frequency does bin k correspond to?" or the reverse.

Step 1 — Frequency resolution: $\Delta f = f_s / N$.

Step 2 — Bin k 's physical frequency (for $k = 0 \dots N/2$): $\text{freq}(k) = k \cdot f_s / N$.

Step 3 — For $k > N/2$ (up to $N-1$), the bin represents a wrapped NEGATIVE frequency: $\text{freq}(k) = (k - N) \cdot f_s / N$.

Worked example

PROBLEM: A 1024-point DFT is computed on a signal sampled at $f_s = 8000$ Hz. What physical frequency does bin $k = 100$ correspond to? What about bin $k = 900$?

Solution:

$\Delta f = 8000/1024 \approx 7.8125$ Hz.

Bin 100 ($\leq N/2 = 512$, so a positive frequency): $\text{freq} = 100 \times 8000/1024 \approx \mathbf{781.25}$ Hz.

Bin 900 (> 512 , so wrapped negative): $\text{freq} = (900 - 1024) \times 8000/1024 = -124 \times 7.8125 \approx \mathbf{-968.75}$ Hz (i.e. it represents a -968.75 Hz component).

Template F — Coding the alias.py functions (your assignment)

Both required functions in your assignment are direct implementations of Template A and basic numerical sampling — no new theory needed, just translate the formulas into NumPy.

lowest_alias_pair(f, fs)

```
def lowest_alias_pair(f, fs):  
    return fs - f
```

That's it — it's exactly Template A's formula. (Given $0 < f < f_s/2$, $f_s - f$ is guaranteed positive and different from f .)

max_sample_difference(f1, f2, fs, duration)

Recreate the sample-time grid exactly as specified ($t = n/fs$), evaluate both cosines on it, and take the max absolute difference elementwise:

```
def max_sample_difference(f1, f2, fs, duration):  
    N = int(duration * fs)  
    n = np.arange(N)  
    t = n / fs  
    x1 = np.cos(2 * np.pi * f1 * t)  
    x2 = np.cos(2 * np.pi * f2 * t)  
    return np.max(np.abs(x1 - x2))
```

Why the difference is $\sim 1e-13$ and not exactly 0: $\cos(2\pi \cdot 700 \cdot t)$ and $\cos(2\pi \cdot (-300) \cdot t + 2\pi \cdot 1000 \cdot t)$ are mathematically identical at the sample points, but floating-point $\cos()$ of a large phase angle (like $2\pi \cdot 700 \cdot 0.099$) accumulates tiny rounding error — that's a computer-precision artifact, not a sign your formula is wrong.

PART 4 — Recommended Practice Problems (no answers given)

Match each problem to a Template from Part 3 before solving. Write down which template you used — that habit alone will make you fast on the actual exam/assignment.

Practice 1 [Template A]

A signal is sampled at $f_s = 2000$ Hz. Find the lowest positive alias partner of $f = 850$ Hz.

Practice 2 [Templates B + A]

$x(t) = 2\cos(2\pi \cdot 1200t) + \cos(2\pi \cdot 3000t)$ is sampled at $f_s = 5000$ Hz. Is this fast enough to avoid aliasing? If not, what alias frequency will the 3000 Hz component fold down to?

Practice 3 [Template C]

After sampling at $f_s = 4000$ Hz, you observe a component at 300 Hz. If you're told the true frequency was between 3500 Hz and 4500 Hz, what was it really?

Practice 4 [Templates B + D]

A microphone signal is band-limited to 20 kHz (edge of human hearing) and sampled at the standard audio rate $f_s = 44100$ Hz. (a) Does this satisfy Nyquist? (b) Specify an ideal reconstruction filter using the conventional cutoff choice.

Practice 5 [Template E]

A 512-point DFT is taken of a signal sampled at $f_s = 16000$ Hz. What physical frequency does bin $k = 64$ correspond to? What about bin $k = 480$?

Practice 6 [Template A (extended — use several values of k)]

Using your completed `alias.py` as a mental model (not code — just reasoning): for $f = 1200$ Hz and $f_s = 3000$ Hz, list THREE different frequencies (not just the lowest one) that would all produce identical samples to $f = 1200$ Hz.

Practice 7 [Conceptual — see Part 1, Section 5]

A signal is sampled at exactly the Nyquist rate ($\omega_s = 2\omega_M$, not strictly greater). Explain in your own words why this boundary case is risky in practice, even though the inequality from the theorem is "satisfied" in the limit.

Practice 8 [Conceptual — see Part 1, Section 7]

You are given $|H_0(j\omega)|$ (the Zero-Order-Hold frequency response) and $|H_1(j\omega)|$ (linear interpolation) side by side. Explain which one leaks more high-frequency content outside the passband, and why, referring to the shape of their time-domain impulse responses (rectangle vs. triangle).

Quick self-check before moving on

- Can you explain, in one sentence, why sampling in time creates copies of the spectrum in frequency?
- Can you state the Nyquist theorem from memory, including which direction the inequality goes?
- Given any f and f_s , can you write down the alias-partner formula without looking it up?
- Do you know the difference between why ZOH and Linear Interpolation have different frequency-domain behavior, just from the shape of their impulse responses (rectangle vs. triangle)?